

Supabase + Stripe Reference

Everything in one place: every Supabase table & storage bucket in this project, plus the full Stripe UPI QR donation integration — commands, code, and the bugs we actually hit.

Contents

Supabase

[hero_banners](#)
[about_section](#)
[portfolio_projects](#)
[faqs](#)
[testimonials](#)
[events / events_section](#)
[gallery_items](#)
[audio_tracks / audio_section](#)
[video_section](#)
[contact_submissions](#)
[donate_section / donations](#)
[donation_tiers \(legacy\)](#)
[site_settings](#)
[profiles](#)

[Storage buckets](#)

[RLS pattern](#)

[Quick REST queries](#)

Stripe / Donate flow

[How the flow works](#)

[Get API keys](#)

[Creating the UPI QR](#)

[Registering the webhook](#)

[Verifying the webhook](#)

[Environment variables](#)

[Stripe CLI — local testing](#)

[Vercel — env vars & deploy](#)

[Gotchas we actually hit](#)

[Test mode → live mode](#)

Supabase — all tables & buckets

This project never used Supabase CLI migrations — every table was created ad hoc via the SQL editor over time. Instead of reconstructing exact historical commands, everything below reflects the **current live schema**, verified by querying Supabase's own schema endpoint (PostgREST's OpenAPI description) and the Storage API directly. The `create table if not exists` statements below will safely reproduce this exact schema if you ever need to recreate it elsewhere.

Naming pattern used across this project

- **Singleton "section" tables** — exactly one row, always `id = 1`, holds the copy/settings for one homepage section (e.g. `about_section`, `donate_section`, `site_settings`). Admin forms always `.update(...).eq("id", 1)`, never insert.
- **Collection tables** — many rows, `uuid` primary key via `gen_random_uuid()`, usually with `is_active` + `sort_order` for admin-controlled visibility/ordering (e.g. `faqs`, `testimonials`, `events`, `portfolio_projects`).

hero_banners collection

```
create table if not exists public.hero_banners (
  id uuid primary key default gen_random_uuid(),
  slug text not null default 'home-hero',
  image_url text not null,
  image_alt text,
  badge_emoji text default '👉',
  badge_text text not null,
  title text not null,
  description text,
  primary_button_text text,
  primary_button_link text,
  secondary_button_text text,
  secondary_button_link text,
  stats jsonb,
  is_active boolean not null default true,
  sort_order integer not null default 0,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Bucket: `hero-banners`

about_section singleton

```
create table if not exists public.about_section (
  id integer primary key,
  image_url text,
  image_url_2 text,
  eyebrow_text text not null default 'About Us',
  title text not null default 'Building Digital Experiences That Inspire',
  description text not null,
  years_experience integer not null default 10,
  button_text text not null default 'Learn More',
  features text[] not null,
  updated_at timestamptz not null default now()
);
```

```
);  
insert into public.about_section (id) values (1) on conflict (id) do nothing;
```

Bucket: about-section

portfolio_projects collection

```
create table if not exists public.portfolio_projects (  
  id uuid primary key default gen_random_uuid(),  
  title text not null,  
  category text not null,  
  image_url text not null,  
  project_link text,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Bucket: portfolio-projects

faqs collection

```
create table if not exists public.faqs (  
  id uuid primary key default gen_random_uuid(),  
  question text not null,  
  answer text not null,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  created_at timestamptz not null default now()  
);
```

No storage bucket (text-only content).

testimonials collection

```
create table if not exists public.testimonials (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  role text not null,  
  image_url text,  
  review text not null,  
  rating smallint not null default 5,
```

```
is_active boolean not null default true,  
sort_order integer not null default 0,  
created_at timestampz not null default now(),  
updated_at timestampz not null default now()  
);
```

Bucket: testimonials

events collection

```
create table if not exists public.events (  
  id uuid primary key default gen_random_uuid(),  
  title text not null,  
  description text not null,  
  event_date date not null,  
  event_time text,  
  location text,  
  link_url text,  
  image_url text,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  created_at timestampz not null default now()  
);
```

Bucket: events

events_section singleton

```
create table if not exists public.events_section (  
  id integer primary key,  
  background_image_url text  
);  
insert into public.events_section (id) values (1) on conflict (id) do nothing;
```

Bucket: events-section (background image only – separate from the events bucket used for individual event images)

gallery_items collection

```
create table if not exists public.gallery_items (  
  id uuid primary key default gen_random_uuid(),  
  media_type text not null, -- 'image' | 'video'  
  image_url text,
```

```
video_url text,  
title text,  
is_active boolean not null default true,  
sort_order integer not null default 0,  
created_at timestamptz not null default now()  
);
```

Bucket: `gallery`

audio_tracks collection

```
create table if not exists public.audio_tracks (  
  id uuid primary key default gen_random_uuid(),  
  title text not null,  
  description text,  
  audio_url text not null,  
  cover_image_url text,  
  is_active boolean not null default true,  
  sort_order integer not null default 0,  
  created_at timestamptz not null default now()  
);
```

audio_section singleton

```
create table if not exists public.audio_section (  
  id integer primary key,  
  background_image_url text  
);  
insert into public.audio_section (id) values (1) on conflict (id) do nothing;
```

Bucket: `audio` (audio files + cover images), `audio-section` (section background)

video_section singleton

```
create table if not exists public.video_section (  
  id integer primary key,  
  video_url text not null default '',  
  title text not null default 'See How We Bring Ideas To Life',  
  description text not null default 'A quick look at how our team designs, builds, and',  
  is_active boolean not null default true,  
  updated_at timestamptz not null default now()  
);
```

```
);  
insert into public.video_section (id) values (1) on conflict (id) do nothing;
```

No storage bucket – `video_url` stores an external link (e.g. YouTube), not an uploaded file.

contact_submissions collection

```
create table if not exists public.contact_submissions (  
  id uuid primary key default gen_random_uuid(),  
  full_name text not null,  
  email text not null,  
  company text,  
  phone text not null,  
  message text not null,  
  created_at timestamptz not null default now()  
);
```

Bucket: `contact-section` (section background image only – submissions themselves have no file uploads)

donate_section singleton

```
create table if not exists public.donate_section (  
  id integer primary key,  
  background_image_url text,  
  title text not null default 'Support Our Mission',  
  subtitle text not null default 'Give Back Today',  
  description text,  
  button_text text not null default 'Donate Now',  
  phone_number text,          -- "Settlement UPI ID (reference only)" field  
  razorpay_qr_code_id text,   -- legacy, unused since the Stripe switch  
  qr_code_image_url text,     -- legacy, unused since the Stripe switch  
  default_amount numeric not null default 500  
);  
insert into public.donate_section (id) values (1) on conflict (id) do nothing;
```

Bucket: `donate-section`

donations collection

```
create table if not exists public.donations (  
  id uuid primary key default gen_random_uuid(),
```

```

razorpay_order_id text,          -- legacy, unused since the Stripe switch
razorpay_payment_id text unique, -- legacy, unused since the Stripe switch
stripe_payment_intent_id text unique,
amount numeric not null,
currency text not null default 'INR',
status text not null default 'created',
donor_name text,
donor_email text,
donor_contact text,
created_at timestamptz not null default now()
);

alter table public.donations enable row level security;

drop policy if exists "Admins can read donations" on public.donations;
create policy "Admins can read donations"
  on public.donations for select
  using ((auth.jwt() -> 'app_metadata' ->> 'role') = 'admin');

```

Written only by the Stripe webhook via the service-role key (bypasses RLS) — no client-side insert policy needed. No storage bucket. See the [Stripe section](#) below for how this table gets populated.

donation_tiers legacy / unused

This table exists in the database but is **not referenced anywhere in the current codebase** — it appears to be left over from an earlier donation-page design (fixed donation tiers/amounts) before the QR-based Stripe flow. Safe to drop if you've confirmed nothing external depends on it.

```

-- Reconstructed schema, for reference only:
create table if not exists public.donation_tiers (
  id uuid primary key default gen_random_uuid(),
  slug text not null,
  title text not null,
  amount text not null,
  description text,
  impact text[] not null,
  is_featured boolean not null default false,
  button_text text,
  donate_url text,
  sort_order integer not null default 0,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

```

site_settings singleton

```
create table if not exists public.site_settings (  
  id integer primary key,  
  contact_phone text,  
  contact_email text,  
  office_address text,  
  working_hours text,  
  facebook_url text,  
  instagram_url text,  
  twitter_url text,  
  linkedin_url text,  
  contact_background_image_url text,  
  show_login_button boolean not null default true,  
  updated_at timestampz default now()  
);  
insert into public.site_settings (id) values (1) on conflict (id) do nothing;
```

Used for the dashboard's "show/hide public Login button" toggle and the Contact section's details.

profiles auth

```
create table if not exists public.profiles (  
  id uuid primary key references auth.users(id) on delete cascade,  
  email text not null,  
  full_name text,  
  role text not null default 'user', -- 'user' | 'admin'  
  created_at timestampz not null default now(),  
  updated_at timestampz not null default now()  
);
```

The actual admin gate used everywhere (proxy.ts middleware, and every RLS policy in this project) reads auth.jwt() -> 'app_metadata' ->> 'role' — **not** this table's role column directly. app_metadata lives on the Supabase Auth user record itself (settable only via the Admin API / dashboard, never by the user), so profiles.role is a readable mirror for the UI, while app_metadata.role is the actual source of truth enforced by every policy and by the middleware's route protection.

Storage buckets — all 11, all public

hero-banners

about-section

portfolio-projects

hero_banners images	about_section images	portfolio_projects images
testimonials testimonial photos	events event images	events-section events section background
gallery gallery_items media	audio audio_tracks files + covers	audio-section audio section background
donate-section donate_section background	contact-section contact section background	

Every bucket in this project is created the same way (Dashboard → Storage → New bucket → toggle **Public** on, no file size limit or MIME restriction set) — or equivalently via SQL:

```
insert into storage.buckets (id, name, public)
values ('bucket-name', 'bucket-name', true)
on conflict (id) do nothing;
```

Public buckets mean anyone with the URL can view the file (fine for CMS marketing images) — but **uploads** still only happen through admin-authenticated dashboard forms, since the upload calls run client-side using the logged-in admin's session, not anonymous access.

RLS pattern used across this project

Two flavors, depending on whether the public homepage needs to read the table:

Public-read CMS content (hero_banners, about_section, faqs, testimonials, events, portfolio_projects, gallery_items, audio_tracks, audio_section, video_section, donate_section, site_settings, events_section):

```
alter table public.<table> enable row level security;

create policy "Public can read"
  on public.<table> for select
  using (true);

create policy "Admins can write"
  on public.<table> for all
  using ((auth.jwt() -> 'app_metadata' ->> 'role') = 'admin')
  with check ((auth.jwt() -> 'app_metadata' ->> 'role') = 'admin');
```

Admin-only content (donations, contact_submissions):

```
alter table public.<table> enable row level security;

create policy "Admins can read"
  on public.<table> for select
  using ((auth.jwt() -> 'app_metadata' ->> 'role') = 'admin');

-- contact_submissions also needs public INSERT (the contact form is
-- filled out by anonymous visitors), donations does not (only the
-- service-role webhook ever writes to it, which bypasses RLS entirely):
create policy "Anyone can submit"
  on public.contact_submissions for insert
  with check (true);
```

Quick REST queries (curl) — works for any table above

```
SUPABASE_URL="https://your-project.supabase.co"
SERVICE_ROLE_KEY="..." # Project Settings → API → service_role (server-only, bypasses RLS)

# Read
curl -s "$SUPABASE_URL/rest/v1/<table>?select=*&order=created_at.desc&limit=10" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"

# Update a singleton section row
curl -s -X PATCH "$SUPABASE_URL/rest/v1/<table>?id=eq.1" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY" \
  -H "Content-Type: application/json" -H "Prefer: return=representation" \
  -d '{"field": "value"}'

# Delete a row
curl -s -X DELETE "$SUPABASE_URL/rest/v1/<table>?id=eq.<row-id>" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"

# List storage buckets
curl -s "$SUPABASE_URL/storage/v1/bucket" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"

# Inspect the FULL live schema (every table + column, straight from PostgREST)
curl -s "$SUPABASE_URL/rest/v1/" \
  -H "apikey: $SERVICE_ROLE_KEY" -H "Authorization: Bearer $SERVICE_ROLE_KEY"
```

Always use the **service_role** key for these — never the anon key, and never from browser-side code. It bypasses RLS entirely.

Stripe — the Donate (UPI QR) integration

A donor clicks **Donate Now** on the homepage. The browser calls our own API route, which asks Stripe to create a **PaymentIntent** for a fixed, admin-configured amount (`donate_section.default_amount`) using the **UPI** payment method with a QR display flow. Stripe returns a hosted QR image URL, which we show in a popup. When the donor actually pays (scanning with a real UPI app in live mode), Stripe fires a **webhook** to our server, which verifies it and writes a row into the `donations` table.

```
Browser → POST /api/donate/stripe-qr → Stripe PaymentIntents API → QR image URL
Donor scans QR (real money only in live mode)
Stripe → POST /api/webhooks/stripe → verify signature → insert into Supabase `donations`
Admin → /admin/donate → SELECT * FROM donations
```

Get your API keys (Dashboard)

1. Stripe Dashboard → **Developers** → **API keys**
2. Toggle **Test mode** while developing (top-right switch)
3. Copy the **Secret key** → `STRIPE_SECRET_KEY` (`sk_test_...` for test, `sk_live_...` for real payments)

UPI eligibility: Stripe only supports UPI as a payment method for India-registered accounts with UPI explicitly enabled. Confirm this in Dashboard → Settings → Payment methods before going live.

Creating the UPI QR PaymentIntent

`app/api/donate/stripe-qr/route.ts` — the actual call that generates the QR:

```
const paymentIntent = await stripe.paymentIntents.create({
  amount: Math.round(amount * 100), // rupees → paise
  currency: "inr",
  payment_method_types: ["upi"],
  payment_method_data: {
    type: "upi",
    billing_details: {
      name: "Donor",
      address: {
        country: "IN",
        line1: "NA",
        city: "NA",
      },
    },
  },
})
```

```

        state: "NA",
        postal_code: "110001",
      },
    },
    payment_method_options: { upi: { flow: "qr" } },
    confirm: true,
  });

// The QR image lives here – NOT under `upi_display_qr_code` as older
// docs/examples suggest. Confirmed empirically against the live API:
const imageUrl = paymentIntent.next_action
  ?.upi_handle_redirect_or_display_qr_code
  ?.qr_code
  ?.image_url_png;

```

Registering a webhook endpoint

Stripe won't send any events until an endpoint is registered for your *current mode* (test or live — they're separate). Two ways to do it:

Dashboard: Developers → Webhooks → Add endpoint → URL `https://your-domain/api/webhooks/stripe` → event `payment_intent.succeeded` → copy the **Signing secret**.

Or via the API directly (what we used for this project):

```

curl -s https://api.stripe.com/v1/webhook_endpoints -u "$STRIPE_SECRET_KEY:" \
  -d "url=https://supabase-learning-animations.vercel.app/api/webhooks/stripe" \
  -d "enabled_events[]=payment_intent.succeeded"

# Response includes the signing secret, e.g.
# "secret": "whsec_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
# → this becomes STRIPE_WEBHOOK_SECRET

```

List existing endpoints (useful to check none are missing/duplicated):

```

curl -s https://api.stripe.com/v1/webhook_endpoints -u "$STRIPE_SECRET_KEY:"

```

Verifying the webhook

`app/api/webhooks/stripe/route.ts` — must read the **raw** body (not parsed JSON):

```

const rawBody = await request.text();
const signature = request.headers.get("stripe-signature");
const event = stripe.webhooks.constructEvent(rawBody, signature, webhookSecret);

if (event.type === "payment_intent.succeeded") {
  const paymentIntent = event.data.object;
  const supabase = createAdminClient();
  await supabase.from("donations").upsert({
    stripe_payment_intent_id: paymentIntent.id,
    amount: paymentIntent.amount / 100,
    currency: paymentIntent.currency.toUpperCase(),
    status: "captured",
    donor_email: paymentIntent.receipt_email ?? null,
  }, { onConflict: "stripe_payment_intent_id" });
}

```

Environment variables

Variable	Where it's used	Where to get it
NEXT_PUBLIC_SUPABASE_URL	All Supabase clients	Supabase → Project Settings → API
NEXT_PUBLIC_SUPABASE_ANON_KEY	Browser-side client	Supabase → Project Settings → API
SUPABASE_SERVICE_ROLE_KEY	Webhook DB writes only	Supabase → Project Settings → API
STRIPE_SECRET_KEY	Creating PaymentIntents	Stripe → Developers → API keys
STRIPE_WEBHOOK_SECRET	Verifying webhook signatures	Stripe webhook endpoint's <code>secret</code> field

Local vs. production are separate! `.env.local` only affects `npm run dev` on your machine. Vercel Production needs its own copies added via `vercel env add` — see below.

Stripe CLI — local testing

```

# One-time login (opens browser to authorize the CLI)
stripe login

# Check which account/keys the CLI is using

```

```
stripe config --list

# Forward Stripe events to your local dev server. Prints a signing secret –
# use THIS one in local .env.local, not the Dashboard endpoint's secret.
stripe listen --forward-to localhost:3000/api/webhooks/stripe

# In a second terminal: simulate a successful payment without a real card/UPI app
stripe trigger payment_intent.succeeded

# Look up API/docs reference from the terminal
stripe docs search "upi payment method"
stripe docs api "POST /v1/payment_intents"
```

There's an npm script wired up for convenience: `npm run stripe runs stripe listen --forward-to localhost:3000/api/webhooks/stripe .`

Vercel — env vars & deploy

```
# See what's currently set on Production
vercel env ls production

# Add a var (pipe the value in so it's non-interactive)
printf '%s' 'sk_test_xxxxx' | vercel env add STRIPE_SECRET_KEY production

# Remove a stale one before replacing it
vercel env rm STRIPE_WEBHOOK_SECRET production --yes

# Pull production env vars down locally (alternative to manually copying)
vercel env pull --yes

# Deploy to production
vercel --prod --yes

# Check recent function logs (great for debugging a live 500)
vercel logs https://your-domain --json
```

Alias quirk on this project: every `vercel --prod run` re-attaches an old alias (`g-sap-animation-alpha.vercel.app`) automatically. After every deploy:

```
vercel alias remove g-sap-animation-alpha.vercel.app --yes
vercel alias set <newest-deployment-url> supabase-learning-animations.vercel.app
```

Gotchas we actually hit (and fixed)

- **Missing `billing_details.name`** : Stripe rejects a UPI PaymentIntent without it.
- **Missing `billing_details.address`** : a later requirement — the whole address object (not just `country`) is needed, even though the general API schema lists every address field as "optional." This is enforced dynamically for UPI specifically, not documented in the generic reference.
- **Wrong response field name:** the QR image is under `next_action.upi_handle_redirect_or_display_qr_code.qr_code.image_url_png` — not `upi_display_qr_code.image_data_url` as older examples suggest. Confirmed by logging the raw `next_action` object.
- **Minimum charge amount:** Stripe rejects INR amounts under roughly **\$0.50 USD equivalent** (fluctuates with exchange rate — `amount_too_small` error). We enforce a ₹50 floor in the Zod schema, the admin form, and the API route itself so this fails fast with a clean message instead of a raw Stripe error.
- **No webhook endpoint registered:** easy to miss — without one, Stripe has nowhere to send events in production, even if your code and env vars are perfect. Always confirm with `curl https://api.stripe.com/v1/webhook_endpoints -u "$STRIPE_SECRET_KEY:"` .
- **Local `stripe listen` secret $\neq$ production webhook secret.** They're two different signing secrets for two different registered destinations. Don't reuse one for the other.

Switching test mode → live mode

1. Stripe Dashboard → toggle **off** Test mode
2. Developers → API keys → copy the `sk_live_...` secret key
3. Developers → Webhooks → add a **new** endpoint (live-mode endpoints are separate from test-mode ones) → copy its `whsec_...`
4. Update `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` on Vercel Production with the live values
5. Redeploy

Do this deliberately, not by accident — live mode moves real money. Double-check the key prefix (`sk_live_` vs `sk_test_`) before deploying.

Generated for this project – the Supabase section reflects the live database schema (introspected directly, not recalled from memory), and the Stripe section reflects the working, verified Donate integration as of the last deploy.